

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2020 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Varianta 3 (22 iulie 2020)	2
SUBIECTUL I (60 de puncte)	2
1. Programare Orientată pe Obiecte (POO): Încapsulare	2
2. Structura sistemelor de calcul: Dispozitive periferice	2
3. Programare: Încadrare cifre în baze de numerație	3
4. Baze de date: Școală de dans	4
SUBIECTUL al II-lea (30 de puncte)	5
1. Activitate didactică pentru secvența A: pseudocod și structură repetitivă	5
2. Itemi de tip eseu	6

I. Rezolvare Definitivat Varianta 3 (22 iulie 2020)

[Subiect PDF](file:

SUBIECTUL I (60 de puncte)

1. Programare Orientată pe Obiecte (POO): Încapsulare

- **Încapsulare:** Principiul POO ce ascunde starea internă a unui obiect și restricționează accesul direct la membrii săi prin definirea de metode publice (getter/setter) de acces.
- **Clase și Obiecte:** Clasa reprezintă șablonul/tipul de date, iar obiectul este instanța fizică a clasei alocată în memorie.
- **Niveluri de acces:**
 1. public: Accesibil de oriunde din program.
 2. private: Accesibil doar din interiorul aceleiași clase.
 3. protected: Accesibil din clasă și din clasele derivate (moștenite).
- **Constructorii/Destructorii:** Metode speciale apelate la crearea, respectiv distrugerea automată a unui obiect.
- **Problemă (Gestiune cont bancar): Enunț:** Se cere modelarea unui cont bancar printr-o clasă care reține titularul și soldul. Soldul nu trebuie modificat direct din exterior, ci numai prin metode care validează operațiile.

Descriere: Clasa ContBancar are membrii de date private, deci starea obiectului este protejată. Constructorul inițializează titularul și soldul, metoda depune modifică soldul numai dacă suma este pozitivă, iar metoda getSold permite citirea controlată a valorii. Astfel se respectă principiul încapsulării.

► C++:

```
#include <iostream>
#include <string>
using namespace std;
class ContBancar {
private:
    string titular;
    double sold;
public:
    ContBancar(string t, double s) : titular(t), sold(s) {}
    void depune(double suma) { if(suma > 0) sold += suma; }
    double getSold() { return sold; }
};
int main() {
    ContBancar cont("Daniel", 1000);
    cont.depone(500);
    cout << "Sold curent: " << cont.getSold() << endl;
    return 0;
}
```

2. Structura sistemelor de calcul: Dispozitive periferice

- **Integrare:** Perifericele permit introducerea de date (intrare) și redarea rezultatelor (ieșire), comunicând cu procesorul prin controlere speciale de I/O și întreruperi software/hardware.

- **Interacțiune periferic-procesor:** La apăsarea unei taste, controlerul tastaturii codifică semnalul, îl transmite către sistem și poate genera o întrerupere. Procesorul întrerupe temporar execuția curentă, sistemul de operare preia codul tastei prin driver, îl plasează în bufferul de intrare, iar aplicația activă îl citește și îl interpretează. Pentru un dispozitiv de ieșire, cum este imprimanta, procesorul transmite datele către driver, driverul le convertește în comenzi specifice dispozitivului, iar controlerul imprimantei execută tipărirea.
- **Clasificare:**
 1. **Periferece de intrare:** Mouse, Tastatură (parametru: DPI pentru mouse, polling rate).
 2. **Periferece de ieșire:** Monitor, Imprimantă (parametru: rezoluție PPI/DPI, timp de răspuns).
 3. **Periferece de intrare/ieșire (I/O):** Ecran tactil, placă de rețea.

3. Programare: Încadrare cifre în baze de numerație

- **Descriere:** Cifra maximă a fiecărui număr determină baza minimă în care acesta poate fi scris (baza minimă = $cMax(n) + 1$). Programul citește șirul, determină cifra maximă pentru fiecare termen și actualizează contoarele pentru toate bazele superioare acelei cifre maxime.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int cMax(long long n) {
    int mx = 0;
    long long temp = n;
    while (temp > 0) {
        int cif = temp % 10;
        if (cif > mx) mx = cif;
        temp /= 10;
    }
    return mx;
}

int main() {
    ifstream fin("def2020.in");
    long long x;
    int baze[11] = {0}; // contoare pentru baze de la 2 la 10

    while (fin >> x) {
        int mx = cMax(x);
        // Numărul poate fi scris în orice bază b > mx
        for (int b = mx + 1; b <= 10; ++b) {
            if (b >= 2) baze[b]++;
        }
    }
    fin.close();

    for (int b = 2; b <= 10; ++b) {
        cout << baze[b] << (b == 10 ? "" : " ");
    }
    cout << endl;
}
```

```
    return 0;
}
```

Soluție Pascal:

```
program BazeNumeratie;
var
    fin: text;
    x: int64;
    baze: array[2..10] of integer;
    b, mx, i: integer;

function cMax(n: int64): integer;
var mx, cif: integer;
begin
    mx := 0;
    while n > 0 do
    begin
        cif := n mod 10;
        if cif > mx then mx := cif;
        n := n div 10;
    end;
    cMax := mx;
end;

begin
    for b := 2 to 10 do baze[b] := 0;
    assign(fin, 'def2020.in');
    reset(fin);

    while not eof(fin) do
    begin
        read(fin, x);
        mx := cMax(x);
        for b := mx + 1 to 10 do
        begin
            if b >= 2 then
                baze[b] := baze[b] + 1;
            end;
        end;
        close(fin);

        for b := 2 to 10 do
        begin
            write(baze[b]);
            if b < 10 then write(' ');
        end;
        writeln;
    end.
end.
```

4. Baze de date: Școală de dans

Model conceptual:

- CURSANT: id_cursant (PK), nume, prenume, data_nasterii.
- STIL_DANS: id_stil (PK), nume_stil, categoria, descriere.

- INSCRIERE: id_cursant (FK), id_stil (FK), an_inscriere, PK este (id_cursant, id_stil, an_inscriere).

Relații și normalizare: CURSANT și STIL_DANS sunt în relație M:N prin INSCRIERE, deoarece un cursant poate urma mai multe stiluri, iar un stil poate avea mai mulți cursanți. Cheia compusă (id_cursant, id_stil, an_inscriere) identifică înscrierea pentru un an. Modelul respectă 1NF prin valori atomice, 2NF prin separarea atributelor dependente de cursant/stil, și 3NF prin evitarea dependențelor tranzitive. Cheile străine trebuie să existe înainte de inserarea înscrierilor.

Model fizic:

```
CREATE TABLE CURSANT (  
    id_cursant INT PRIMARY KEY,  
    nume VARCHAR(50) NOT NULL,  
    prenume VARCHAR(50) NOT NULL,  
    data_nasterii DATE NOT NULL  
);  
  
CREATE TABLE STIL_DANS (  
    id_stil INT PRIMARY KEY,  
    nume_stil VARCHAR(60) NOT NULL,  
    categoria VARCHAR(40) NOT NULL,  
    descriere VARCHAR(255)  
);  
  
CREATE TABLE INSCRIERE (  
    id_cursant INT NOT NULL,  
    id_stil INT NOT NULL,  
    an_inscriere INT NOT NULL,  
    PRIMARY KEY (id_cursant, id_stil, an_inscriere),  
    FOREIGN KEY (id_cursant) REFERENCES CURSANT(id_cursant),  
    FOREIGN KEY (id_stil) REFERENCES STIL_DANS(id_stil)  
);
```

SQL pentru adăugarea stilului cerut:

```
INSERT INTO STIL_DANS (nume_stil, categoria, descriere)  
VALUES ('tango argentinian', 'clasic', 'Dans inclus pe lista patrimoniului UNESCO');
```

—

SUBIECTUL al II-lea (30 de puncte)

1. Activitate didactică pentru secvența A: pseudocod și structură repetitivă

Momente esențiale ale lecției: organizarea clasei; actualizarea cunoștințelor despre structuri de bază; predarea/dirijarea învățării; fixarea și evaluarea formativă.

Activități de învățare:

1. Identificarea pașilor de rezolvare pentru calculul sumei cifrelor unui număr.
2. Reprezentarea algoritmului în pseudocod folosind structura repetitivă cât timp.

Metodă: conversația euristică și exercițiul. Profesorul pornește de la un exemplu numeric, cere elevilor să extragă ultima cifră prin $n \bmod 10$, apoi să elimine cifra prin $n \div 10$. Elevii scriu pseudocodul, îl testează manual și corectează condiția de oprire $n > 0$.

2. Itemi de tip eseu

Caracteristici: permit răspunsuri dezvoltate; evaluează argumentarea și organizarea ideilor; pot avea barem analitic. **Regulă:** cerința trebuie să precizeze clar tema, reperele și criteriile de evaluare. **Categorii:** eseu structurat și eseu liber.

Item A: Redactați un eseu structurat despre rolul pseudocodului în reprezentarea algoritmilor, având în vedere: claritatea pașilor, independența față de limbajul de programare, utilizarea structurii repetitive. **Criterii:** corectitudine 5p, exemple 5p, organizare 5p.

Item B: Redactați un eseu despre avantajele lucrului în rețea, referindu-vă la partajarea resurselor, comunicare și tipuri de rețele. **Criterii:** concepte corecte 6p, exemple 5p, coerența argumentării 4p.